

{creative} [coding]

Dokumentation Woche 4

{Montag} 18.05.2026

Vormittag

- Zwischenpräsentationen
 - präsentieren der aktuellen Code-Snippets
 - Feedback geben an lara & Luka

Nachmittag

- Feedbackgespräch mit Stefanie und Yann
 - Writing-Assessment zur Mid-Term-Presentation abschliessen
-

{Dienstag} 19.05.2026

Exkursion nach Basel

- Aufgrund eines Arzttermines, konnte ich leider nicht anwesend sein.
-

{Mittwoch} 20.05.2026

Vormittag

- Selbststudium
- Feedback analysieren, weiteres Vorgehen erarbeiten

Nachmittag

- Theorie-Input von Stefanie
 - Literatur lesen und Peer-to-Peer teaching um Inhalte zu vermitteln
 - World of Art
 - Weibliche Elektronik oder eine andere Geschichte elektronischer Musik von Claudia Tittel
-

{Donnerstag} 21.05.2026

Vormittag

- Selbststudium
- Flock-Code versuchen zu verstehen

Nachmittag

- Selbststudium
- Experimente mit unterschiedlichen Formen

☆ KLEINER SCHWARM

```
// _____
// Globale Variablen & p5.js-Einstiegspunkte
// _____

// noprotect – sagt dem p5.js-Editor: "Bitte nicht eingreifen,
//          auch wenn eine Schleife lang läuft."
let flock; // Wird später mit einem Flock-Objekt befüllt

// setup() wird von p5.js EINMAL beim Start aufgerufen
function setup() {
  createCanvas(windowWidth, windowHeight); // Zeichenfläche = ganzes Browserfenster
  flock = new Flock();                     // Leerer Schwarm wird angelegt
  for (let i = 0; i < 100; i++)
    flock.add(new Boid(width/2, height/2)); // Anz. Boids starten in Mitte definieren
}

// draw() wird von p5.js ca. 60× pro Sekunde aufgerufen – jeder Aufruf = 1 Frame
function draw() {
  background(82, 0, 69); // Hintergrund-Farbe (löscht das vorherige Frame)
  flock.run();           // Jeden Boid bewegen und zeichnen
}

// Wenn die Maus gedrückt und gezogen wird, erscheinen neue Boids
function mouseDragged() {
  flock.add(new Boid(mouseX, mouseY)); // Neuer Boid genau an der Mausposition
}

// _____
// Klasse Flock – der Schwarm
// _____

// Flock ist nur ein Behälter: eine Liste aller Boids plus zwei Hilfsmethoden.
// Der Schwarm selbst "denkt" nicht – das macht jeder Boid für sich.
class Flock {
  constructor() {
    this.boids = []; // Leeres Array, das alle Boid-Objekte hält
  }
  add(b) { this.boids.push(b); } // Einen neuen Boid zur Liste hinzufügen
  run()   {
```

```

    // Jeden Boid in der Liste einmal ausführen.
    // Jeder bekommt die GESAMTE Liste (this.boids), damit er
    // seine Nachbarn selbst herausfiltern kann.
    this.boids.forEach(b => b.run(this.boids));
  }
}

// =====
// Klasse Boid – ein einzelnes Tier im Schwarm
// =====

class Boid {
  constructor(x, y) {
    // pos = aktuelle Position auf der Leinwand (x/y-Koordinaten)
    this.pos = createVector(x, y);
    // vel = Geschwindigkeit: wohin und wie schnell bewegt er sich gerade?
    // Zufällige Startwerte → jeder Boid fliegt am Anfang in eine andere Richtung
    this.vel = createVector(random(-1,1), random(-1,1));
    // acc = Beschleunigung: welche Kraft wirkt gerade auf ihn?
    // Startet immer bei (0,0) – kein Schub am Anfang
    this.acc = createVector(0, 0);
    this.r = 1; // r = Radius/Größe des Dreiecks in Pixeln
    this.ms = 3; // ms = max speed – wie schnell darf er maximal sein?
    this.mf = 0.03; // mf = max force – wie stark darf er seine Richtung ändern?
    // (kleiner Wert = träges, realistisches Lenken)
  }

  // run() wird jeden Frame einmal aufgerufen.
  // bs = das Array aller Boids (um Nachbarn zu finden)
  run(bs) {
    // — Schritt 1: Kräfte berechnen und aufaddieren —
    this.acc.add(this.separate(bs).mult(1.2)); // Abstand halten (1.5× stärker gewichtet)
    this.acc.add(this.align(bs)); // Gleiche Richtung wie Nachbarn
    this.acc.add(this.cohesion(bs)); // Zum Zentrum der Nachbarn bewegen

    // — Schritt 2: Physik anwenden —
    this.vel.add(this.acc).limit(this.ms); // Geschwindigkeit += Beschleunigung, aber max. ms
    this.pos.add(this.vel); // Position += Geschwindigkeit
    this.acc.set(0, 0); // Beschleunigung auf 0 zurücksetzen (wird nächsten Frame
    neu berechnet)

    // — Schritt 3: Zeichnen —
    this.borders(); // Rand-Teleport: am rechten Rand raus = links wieder rein
    this.render(); // Dreieck auf die Leinwand zeichnen
  }

  // — Hilfsmethode: Nachbarn im Radius finden —
  // dist = wie weit schaut der Boid?
  // fn = was soll mit jedem Nachbar gemacht werden? (als Funktion übergeben)
  // Gibt den Durchschnittsvektor aller gefundenen Nachbarn zurück.
  steer(bs, dist, fn) {
    let sum = createVector(0, 0); // Hier werden die Vektoren aufaddiert
    let count = 0; // Zählt, wie viele Nachbarn gefunden wurden
    for (let o of bs) {
      let d = p5.Vector.dist(this.pos, o.pos); // Abstand zu diesem Boid berechnen
      // d > 0 schließt den Boid selbst aus (Abstand zu sich selbst = 0)
      if (d > 0 && d < dist) { fn(sum, o, d); count++; }
    }
  }
}

```

```

    }
    // Durchschnitt zurückgeben, oder Nullvektor wenn keine Nachbarn gefunden
    return count > 0 ? sum.div(count) : createVector(0, 0);
}

// Normalisiert einen Vektor zu einer Lenkkraft:
// "Ich will in Richtung s, mit Maximalgeschwindigkeit, abzüglich meiner aktuellen Fahrtrichtung"
// Das Ergebnis ist die nötige Kurskorrektur.
limit(s) {
    return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
}

// Berechnet die Lenkkraft in Richtung eines Zielpunkts t
seek(t) {
    return this.limit(p5.Vector.sub(t, this.pos)); // Richtungsvektor zum Ziel berechnen
}

// — REGEL 1: Separation —————
// "Halte Abstand!" — Boids die zu nah sind, stoßen sich ab.
// Radius 25 px: nur sehr nahe Nachbarn zählen.
// Je näher der Nachbar, desto stärker die Abstoßung (÷ d).
separate(bs) {
    let s = this.steer(bs, 25, (sum, o, d) =>
        sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d)));
    // Nur lenken wenn wirklich Nachbarn da sind (mag() = Länge des Vektors)
    return s.mag() > 0 ? this.limit(s) : s;
}

// — REGEL 2: Ausrichtung —————
// "Flieg in dieselbe Richtung wie deine Nachbarn!"
// Radius 50 px: weiter schauen als bei Separation.
// Mittelt die Geschwindigkeitsvektoren (vel) aller Nachbarn.
align(bs) {
    let s = this.steer(bs, 50, (sum, o) => sum.add(o.vel));
    return s.mag() > 0 ? this.limit(s) : s;
}

// — REGEL 3: Kohäsion —————
// "Bleib beim Schwarm!" — Bewege dich zur Mitte deiner Nachbarn.
// Radius 50 px. Mittelt die Positionen (pos) aller Nachbarn,
// dann wird seek() benutzt um dorthin zu steuern.
cohesion(bs) {
    let s = this.steer(bs, 50, (sum, o) => sum.add(o.pos));
    return s.mag() > 0 ? this.seek(s) : s;
}

// Rand-Teleport: verlässt ein Boid die Leinwand, kommt er auf der anderen Seite rein.
// Die for-Schleife macht das kompakt für x und y gleichzeitig.
borders() {
    for (let [ax, max] of [['x', width], ['y', height]]) {
        if (this.pos[ax] < -this.r) this.pos[ax] = max + this.r; // Links raus → rechts rein
        if (this.pos[ax] > max + this.r) this.pos[ax] = -this.r; // Rechts raus → links rein
    }
}

// —————
// Den Boid als kleines Dreieck zeichnen, das in Flugrichtung zeigt.

```

```
// -----

render() {
  push(); // Aktuellen Zustand des Koordinatensystems
speichern
  translate(this.pos.x, this.pos.y); // Nullpunkt auf die Boid-Position verschieben
  rotate(this.vel.heading() + radians(90)); // In Flugrichtung drehen (+90° weil Dreieck nach
oben zeigt)
  fill(255, 217, 249); // Füllfarbe: helles Rosa
  stroke(252, 121, 230); // Umrandungsfarbe: kräftiges Pink
  beginShape();
    vertex( 0, -this.r * 2); // Spitze (oben/vorne)
    vertex(-this.r, this.r * 2); // Linke untere Ecke
    vertex( this.r, this.r * 2); // Rechte untere Ecke
  endShape(CLOSE);
  pop(); // Koordinatensystem wieder zurücksetzen (für den nächsten Boid)
}
}
```

☆ WÖRTER SCHWARM

```
// — Wort-Schwarm —
// Wörter in setup() eintragen oder live per addWord("...") hinzufügen
// Maus ziehen = zufälliges Wort aus dem Pool spawnen

let flock;
let wordPool = []; // { word, brightness }
let started = false;

// Graustufen: erstes Wort = hell (220), letztes = dunkel (60)
function brightnessForIndex(i, total) {
  if (total <= 1) return 200;
  return Math.round(220 - (i / (total - 1)) * 160);
}

function addWord(val) {
  if (!val || wordPool.find(w => w.word === val)) return;

  wordPool.push({ word: val, brightness: 200 });

  // Helligkeiten für alle Wörter neu verteilen
  wordPool.forEach((e, i) => {
    e.brightness = brightnessForIndex(i, wordPool.length);
  });
}
```

```

// Einen neuen Boid in der Bildschirmmitte spawnen
let entry = wordPool.find(w => w.word === val);
let b = new Boid(
  width/2 + random(-100, 100),
  height/2 + random(-100, 100),
  val,
  entry.brightness
);
flock.add(b);

// Helligkeiten aller bestehenden Boids aktualisieren
flock.boids.forEach(b => {
  let e = wordPool.find(w => w.word === b.word);
  if (e) b.brightness = e.brightness;
});

started = true;
}

// — p5.js Einstiegspunkte —————

function setup() {
  createCanvas(windowWidth, windowHeight);
  textFont('monospace');
  flock = new Flock();

  // — Wörter hier eintragen —————
  addWord("Drum & Bass");
  addWord("Windrush-Generation");
  addWord("Wind");
  addWord("Chaos");
  addWord("Ordnung");
  addWord("Nähe");
  addWord("Richtung");
  addWord("Flug");
}

function draw() {
  background(10);

  if (!started) {
    fill(255, 255, 255, 45);
    noStroke();
    textSize(13);
    textAlign(CENTER, CENTER);
    text('addWord("dein Wort") aufrufen um zu starten', width/2, height/2);
    return;
  }

  flock.run();
}

// Maus ziehen: zufälliges Wort aus dem Pool an Mausposition spawnen
function mouseDragged() {
  if (!started || wordPool.length === 0) return;
  let entry = wordPool[floor(random(wordPool.length))];
  flock.add(new Boid(mouseX, mouseY, entry.word, entry.brightness));
}

```

```

}

function windowResized() {
  resizeCanvas(windowWidth, windowHeight);
}

// — Flock —————

class Flock {
  constructor() { this.boids = []; }
  add(b) { this.boids.push(b); }
  run() { this.boids.forEach(b => b.run(this.boids)); }
}

// — Boid —————

class Boid {
  constructor(x, y, word, brightness) {
    this.pos = createVector(x, y);
    this.vel = p5.Vector.random2D().mult(random(0.8, 1.6));
    this.acc = createVector(0, 0);
    this.ms = random(1.8, 2.6); // max speed
    this.mf = 0.05; // max force — weich = flüssige Kurven
    this.word = word;
    this.brightness = brightness;
  }

  run(bs) {
    // Separation schwächer, Ausrichtung + Kohäsion stärker → enger aber flexibler Schwarm
    this.acc.add(this.separate(bs).mult(1.0));
    this.acc.add(this.align(bs).mult(1.4));
    this.acc.add(this.cohesion(bs).mult(1.3));

    this.vel.add(this.acc).limit(this.ms);
    this.pos.add(this.vel);
    this.acc.set(0, 0);

    this.borders();
    this.render();
  }

  // Nachbarn im Radius finden und fn darauf anwenden, Durchschnitt zurückgeben
  steer(bs, dist, fn) {
    let sum = createVector(0, 0), count = 0;
    for (let o of bs) {
      let d = p5.Vector.dist(this.pos, o.pos);
      if (d > 0 && d < dist) { fn(sum, o, d); count++; }
    }
    return count > 0 ? sum.div(count) : createVector(0, 0);
  }

  // Lenkkraft: gewünschte Richtung minus aktuelle Geschwindigkeit, begrenzt auf mf
  steerLimit(s) {
    return s.normalize().mult(this.ms).sub(this.vel).limit(this.mf);
  }

  // Zum Zielpunkt t steuern
  seek(t) {

```

```

    return this.steerLimit(p5.Vector.sub(t, this.pos));
  }

  // Regel 1: Abstand halten – Abstoßung von zu nahen Nachbarn (Radius 45 px)
  separate(bs) {
    let s = this.steer(bs, 45, (sum, o, d) =>
      sum.add(p5.Vector.sub(this.pos, o.pos).normalize().div(d)));
    return s.mag() > 0 ? this.steerLimit(s) : s;
  }

  // Regel 2: Ausrichtung – gleiche Richtung wie Nachbarn (Radius 90 px)
  align(bs) {
    let s = this.steer(bs, 90, (sum, o) => sum.add(o.vel));
    return s.mag() > 0 ? this.steerLimit(s) : s;
  }

  // Regel 3: Kohäsion – zum Mittelpunkt der Nachbarn bewegen (Radius 160 px)
  cohesion(bs) {
    let s = this.steer(bs, 160, (sum, o) => sum.add(o.pos));
    return s.mag() > 0 ? this.seek(s) : s;
  }

  // Rand-Teleport: links raus = rechts rein, oben raus = unten rein
  borders() {
    for (let [ax, max] of [['x', width], ['y', height]]) {
      if (this.pos[ax] < -50) this.pos[ax] = max + 50;
      if (this.pos[ax] > max + 50) this.pos[ax] = -50;
    }
  }

  // Als Text zeichnen – Helligkeit bestimmt die Graustufe
  render() {
    push();
    translate(this.pos.x, this.pos.y);
    noStroke();
    textAlign(CENTER, CENTER);
    fill(this.brightness);
    textSize(11);
    text(this.word, 0, 0);
    pop();
  }
}

```


Vormittag

- Präsentation der Arbeiten von Yann
- Input zu Arrays und Audioreactive in P5Live

Nachmittag

- Selbststudium
- Experimente mit unterschiedlichen Formen

★ SPHERE AUDIOREACTIVE

```
/*
  _audio_analysis // cc teddavis.org 2019-24

  revamp of P5LIVE's Audio snippet, now built-in the background!
  add snippet to other sketches: CTRL + SHIFT + A
*/

function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL)

  // audio stuff now behind the scenes, 'true' makes class vars global
  setupAudio(true) // if empty, use 'a5.' before audio vars below
  a5.ease = .275 // customize ease speed (kleiner wert = smooth)
}

function draw() {€
  /* audio vars: amp, ampL, ampR, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()
  background(17, 0, 255)
  noFill()

  // orange
  stroke(255, 115, 0)
  push()
  translate(0, 20)
  sphere(50 + fftEase[100])
  pop()

  // pink
  stroke(255, 0, 170)
  push()
  translate(50, 50)
  sphere(50 + fftEase[20])
  pop()

  // grün
  stroke(194, 255, 161)
  push()
  translate(-50, 50)
  sphere(50 + fftEase[60])
  pop()
}
```

★ SPHERE & TORUS AUDIOREACTIVE

```
/*
  _audio_analysis // cc teddavis.org 2019-24

  revamp of P5LIVE's Audio snippet, now built-in the background!
  add snippet to other sketches: CTRL + SHIFT + A
*/
function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL)

  // audio stuff now behind the scenes, 'true' makes class vars global
  setupAudio(true) // if empty, use 'a5.' before audio vars below
  a5.ease = .275 // customize ease speed (kleiner wert = smooth)
}

function draw() {
  /* audio vars: amp, ampL, ampR, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()
  background(17, 0, 255)
  noFill()

  // orange
  stroke(255, 115, 0)
  sph(0, fftEase[0] * .72, -50, 50 +fftEase[100])

  // pink
  stroke(255, 156, 238)
  sph(50, 50, 30, 50 +fftEase[20])

  // grün
  stroke(194, 255, 161)
  trs(-50, 50, 100, 50 +fftEase[60], 5)
}

function sph(x, y, z, size) {
  push()
  translate(x, y, z)
  sphere(size)
  pop()
}

function trs(x, y, z, size, rSpeed) {
  push()
  translate(x, y, z)
  rotateY(rSpeed * frameCount * 0.01)
  torus(size)
}
```

```
pop()
}
```

★ 3D AUDIOREACTIVE

```
/*
  _audio_analysis // cc teddavis.org 2019-24

  revamp of P5LIVE's Audio snippet, now built-in the background!
  add snippet to other sketches: CTRL + SHIFT + A
*/
function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL)

  // audio stuff now behind the scenes, 'true' makes class vars global
  setupAudio(true) // if empty, use 'a5.' before audio vars below
  a5.ease = .275 // customize ease speed (kleiner wert = smooth)
}

function draw() {
  /* audio vars: amp, ampL, ampR, ampEase, fft, fftEase, waveform, waveformEase */
  orbitControl()
  updateAudio()
  lights()
  background(17, 0, 255)
  noFill()
  noStroke()
  ambientLight(10)
  ambientMaterial(255, 127, 80)

  // orange
  fill(255, 115, 0)
  sph(0, fftEase[0] * .72, -50, 50 +fftEase[100])

  // pink
  fill(255, 156, 238)
  sph(50, 50, 30, 50 +fftEase[20])

  // grün
  fill(194, 255, 161)
  //specularMaterial(255, 255, 0)
  trs(-50, 50, 100, 50 +fftEase[60], 5)
}

function sph(x, y, z, size) {
  push()
  translate(x, y, z)
```

```

sphere(size)
pop()
}

function trs(x, y, z, size, rSpeed) {
  push()
  translate(x, y, z)
  rotateY(rSpeed * frameCount * 0.01)
  torus(size)
  pop()
}

```

★ 3D WÜRFEL GRID AUDIOREACTIVE

```

/*
  _audio_analysis // cc teddavis.org 2019-24

  revamp of P5LIVE's Audio snippet, now built-in the background!
  add snippet to other sketches: CTRL + SHIFT + A
*/

function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL)

  // audio stuff now behind the scenes, 'true' makes class vars global
  setupAudio(true) // if empty, use 'a5.' before audio vars below
  a5.ease = .275 // customize ease speed (kleiner wert = smooth)
}

function draw() {
  //background(0)
  /* audio vars: amp, ampL, ampR, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()
  orbitControl()
  fill(209, 255, 1)

  //gives objects a 3d look by adding lights and shadows
  lights()
  directionalLight(0, 0, 255, -10, 10, 0)

  let number = 6
  let index = 0

  for(let x = 0; x < number; x++) {
    let posX = map(x, 0, number - 1, -width / 4, width / 4)
    for(let y = 0; y < number; y++) {
      let posY = map(y, 0, number - 1, -width / 4, width / 4)

```

```

    for(let z = 0; z < number; z++) {
      let posZ = map(z, 0, number - 1, -width / 4, width / 4)
      cube(posX, posY, posZ, 20 + fftEase[index%fftEase.length]*0.25, 1)
      index++
    }
  }
}

function sph(x, y, z, size, rSpeed) {
  push()
  translate(x, y, z)
  rotateY(frameCount * 0.01 * rSpeed)
  sphere(size)
  pop()
}

function cube(x, y, z, size, rSpeed) {
  push()
  translate(x, y, z)
  rotateZ(frameCount * 0.01 * rSpeed)
  rotateY(frameCount * 0.01 * rSpeed)
  box(size)
  pop()
}

function trs(x, y, z, size, rSpeed) {
  push()
  translate(x, y, z)
  rotateY(frameCount * 0.01 * rSpeed)
  torus(size, 20)
  pop()
}

```

★ 3D WÜRFEL & TORUS GRID AUDIOREACTIVE

```

/*
  _audio_analysis // cc teddavis.org 2019-24

  revamp of P5LIVE's Audio snippet, now built-in the background!
  add snippet to other sketches: CTRL + SHIFT + A
*/
function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL)
  setupAudio(true)
  a5.ease = .275
}

```

```

function draw() {
  updateAudio()
  orbitControl()
  background(0)
  lights()
  directionalLight(0, 0, 255, -10, 10, 0)

  let number = 6
  let index = 0

  for(let x = 0; x < number; x++) {
    let posX = map(x, 0, number - 1, -width / 4, width / 4)
    for(let y = 0; y < number; y++) {
      let posY = map(y, 0, number - 1, -height / 4, height / 4)
      for(let z = 0; z < number; z++) {
        let posZ = map(z, 0, number - 1, -width / 4, width / 4)
        if(index % 2 === 0) {
          stroke(201, 1, 255)
          fill(209, 255, 1)
          cube(posX, posY, posZ, 20 + fftEase[index % fftEase.length] * 0.25, 1)
        } else {
          trs(posX, posY, posZ, 30 + fftEase[index % fftEase.length] * 0.25, 2)
        }
        index++
      }
    }
  }
}

function sph(x, y, z, size, rSpeed) {
  push()
  translate(x, y, z)
  rotateY(frameCount * 0.01 * rSpeed)
  sphere(size)
  pop()
}

function cube(x, y, z, size, rSpeed) {
  push()
  translate(x, y, z)
  rotateZ(frameCount * 0.01 * rSpeed)
  rotateY(frameCount * 0.01 * rSpeed)
  box(size)
  pop()
}

function trs(x, y, z, size, rSpeed) {
  push()
  stroke(0, 255, 200)    // ← eigene Farbe für trs
  fill(255, 50, 0)      // ← eigene Farbe für trs
  translate(x, y, z)
  rotateY(frameCount * 0.01 * rSpeed)
  torus(size, 20,10,6)
  pop()
}

```

